

scripts/generate-coverage-ledger.sh — User Guide

Last verified: 2026-05-16 (constitution-compliance plan Phase 7)
Inheritance: HelixConstitution §11.4.25 (Full-Automation-Coverage Mandate) + §11.4.18 (script docs)

Overview

Generates the canonical docs/coverage-ledger.yaml — Lava's machine-readable coverage matrix per HelixConstitution §11.4.25.

For every feature module, core module, app, lava-api-go service, and owned-by-us Submodule, the generator emits one row recording:

- The path (e.g. feature/onboarding, core/auth, lava-api-go, submodules/containers)
- The kind (feature | core | app | api | submodule)
- Test counts (unit, integration) discovered on disk
- Challenges that target the row (matched by import / // covers-feature: marker / filename keyword / heuristic broad-flow mapping — same matching rules as scripts/check-challenge-coverage.sh)
- Up to 3 sample test file basenames
- The six §11.4.25 invariants, each with status pass | gap | n/a | "waiver: <reason>" | "pass: <reason>"
- An overall row status (covered | partial | gap)

The six §11.4.25 invariants

Invariant	Mechanical inference	Notes
anti_bluff	pass if challenge_count > 0; else gap	Challenges carry §6.AB falsifiability rehearsals
working_capability	pass if challenge_count > 0; else gap	Challenge tests are the E2E gate per §6.AE
documented_promise		Requires operator-or-reviewer override via

Invariant	Mechanical inference	Notes
	Always defaults to gap	waiver — typically points at a CONSTITUTION.md / README.md / spec
no_open_bugs	Always defaults to gap	Requires operator-or-reviewer override via waiver — typically references closure logs under .lava-ci-evidence/crashlytics-resolved/ per §6.AD.3 equivalence
documented	pass if any of {README.md, CLAUDE.md, AGENTS.md, CONSTITUTION.md} present in the path; else gap	Per-scope §6.AD pointer-block makes this pass for any path under feature/, core/, app/, lava-api-go/, submodules/ (after waivers)
four_layer	pass if unit_test_count > 0 AND challenge_count > 0; else gap	§1 four-layer floor: pre-build + post-build + runtime + paired mutation

Status computation

A row's overall status is derived from the six invariant values:

pass count	gap count	Status
6	0	covered
3+	1+	partial
<3	1+	gap

waiver:* and n/a invariants count as neither pass nor gap. A row that has 3+ pass + 0 gap → covered.

Waiver mechanism

The generator default for invariants 3, 4 is gap because they cannot be mechanically inferred. To override, hand-edit docs/coverage-ledger/waivers.yaml:

```
waivers:
  - path: feature/onboarding
```

```
invariants:
  documented_promise: "pass: feature documented in docs/UX-
    FLOWS.md + Onboarding screen content"
  no_open_bugs: "pass: 2026-05-15 audit – no open Crashlytics
    issues per .lava-ci-evidence/crashlytics-resolved/"
```

Waiver values can be:

- "pass: <evidence>" — operator-or-reviewer affirms pass with positive evidence
- "waiver: <reason>" — explicit waiver with tracked-item reference
- "n/a" — invariant does not apply to this row

The waiver file is hand-edited + version-controlled. Per §6.J anti-bluff, every "pass:" waiver SHOULD cite a concrete evidence trail (closure log, commit SHA, doc path).

Usage

```
# Regenerate the ledger + print gap summary
bash scripts/generate-coverage-ledger.sh
```

```
# Emit YAML to stdout instead of writing the file
bash scripts/generate-coverage-ledger.sh --stdout
```

```
# Write silently (no stdout summary)
bash scripts/generate-coverage-ledger.sh --quiet
```

Determinism

Same git tree → same generated row content. The only varying header field is metadata.generated_at (UTC timestamp). The verifier (scripts/check-coverage-ledger.sh) strips metadata + generated_at before diffing so cosmetic timestamp diffs do not fail the freshness check.

Companion files

- docs/coverage-ledger.yaml — the generated ledger (committed)
- docs/coverage-ledger.waivers.yaml — hand-edited per-row waiver overrides (committed)
- scripts/check-coverage-ledger.sh — verifier (asserts presence + freshness + row coverage)
- tests/check-constitution/test_coverage_ledger.sh — hermetic falsifiability rehearsals (6 fixtures)
- scripts/snapshot-coverage-ledger.sh — LVA-019 per-release snapshot mechanism. Per-release frozen copies of this ledger

are written by `scripts/snapshot-coverage-ledger.sh` to `.lava-ci-evidence/coverage-ledger-snapshots/<version>-<code>.yaml` at each Firebase distribute, mirroring the existing `cycle-coverage-map-<version>-<code>.yaml` per-release snapshot pattern used for the §6.AK device-coverage gate. Wired (advisory, non-blocking) into `scripts/firebase-distribute.sh` immediately after the §6.AK Phase-1 Gate 7 block.

Cross-references

- HelixConstitution Constitution.md §11.4.25 (the mandate)
- docs/plans/2026-05-15-constitution-compliance.md Phase 7
- docs/helix-constitution-gates.md CM-COVERAGE-LEDGER row
- scripts/verify-all-constitution-rules.sh (wraps the verifier in advisory mode per §11.4.32)
- Lava CLAUDE.md §6.AD-debt (full-automation-coverage tracker integration)

Inheritance + classification

- Inheritance: HelixConstitution §11.4.25 (the mandate); §11.4.18 (script docs); §6.J/§6.L (anti-bluff posture of the generator)
- Classification: project-specific (the Lava module list is project-specific; the §11.4.25 ledger mandate is universal per HelixConstitution)

Submodule test counts prune nested gitlinks (added 2026-08-27)

`count_submodule_tests()` derives each submodule's nested gitlinks from `git ls-files --stage` (mode 160000) and passes them to `find` as `-prune` arguments, so a submodule's own test count excludes the tests of third-party projects vendored *inside* it.

Why this was needed, measured

`submodules/helixqa/tools/opensource/` contains **27 nested gitlinks** to third-party projects — docling, appium, chroma, perfetto, skyvern, leakcanary, signoz and others. The unpruned `find` walked straight into them, counting their tests as helixqa's:

```
committed ledger : unit_test_count: 392
unpruned find    : unit_test_count: 763      (763 - 392 = 371,
exactly
```

opensource contributes)

392 is the correct number — a `submodules/*` row means the submodule's own tests, per this generator's own header ("owned-by-us reusable code that ships with the project"). IBM's docling tests are not Lava's coverage.

The determinism violation this closed

763 was not merely wrong, it was **machine state rather than tracked state**. The identical git tree with the identical helixqa pin produced 392 or 763 depending purely on whether someone had run a recursive submodule init. That contradicts this generator's own contract (header point 4: *"Determinism. Same git tree → same generated ledger byte-for-byte"*).

The failure surfaced only after the 29 nested submodules were initialised on 2026-08-26. Regenerating the ledger to 763 would have made the gate green on that machine and red on the next one, or on any fresh clone — the "regenerate until it agrees with itself" trap. The ledger was never stale; the generator was over-counting, and the ledger file was left byte-identical.

Known residual: `count_tests()` — used for `feature/*`, `core/*`, `app` and `lava-api-go` rows — still counts **untracked** files, so those rows remain sensitive to local build and scratch state. No such drift exists today; it is a latent instance of the same class.